

For this assignment, I chose the Fashion-MNIST dataset. Since it consists of image data, it provides a unique challenge for a standard Feed-Forward Neural Network (FFN). Unlike a Convolutional Neural Network (CNN), a fully connected FFN lacks inherent spatial understanding. This makes it an interesting task to see how the model performs using only architectural and preprocessing "tricks."

The dataset consists of thousands of 28×28 grayscale images of clothing, where each image is originally represented as a 1D flattened row of 784 pixel values. Using NumPy, these rows are combined to make up a 2D matrix where each row is a distinct image and each column represents a specific pixel's intensity between 0-255. Before separating the data into training, validation and testing sets the data needs to be normalized in order to not run into the exploding gradient problem. The model provides options for either Z-Score normalization (zero mean, unit variance) or Min-Max scaling to ranges such as $[0,1]$ or $[-1,1]$. For this implementation, I would prefer Min-Max scaling as it aligns more closely with the goal of image normalization. Unlike Z-Score normalization, which centers data around a mean and potentially introduces unnecessary changes based on variance, Min-Max scaling preserves the relative intensity of the pixels while ensuring they fall within a stable, bounded range. But Z-score converges faster so it's the one used in most tests.

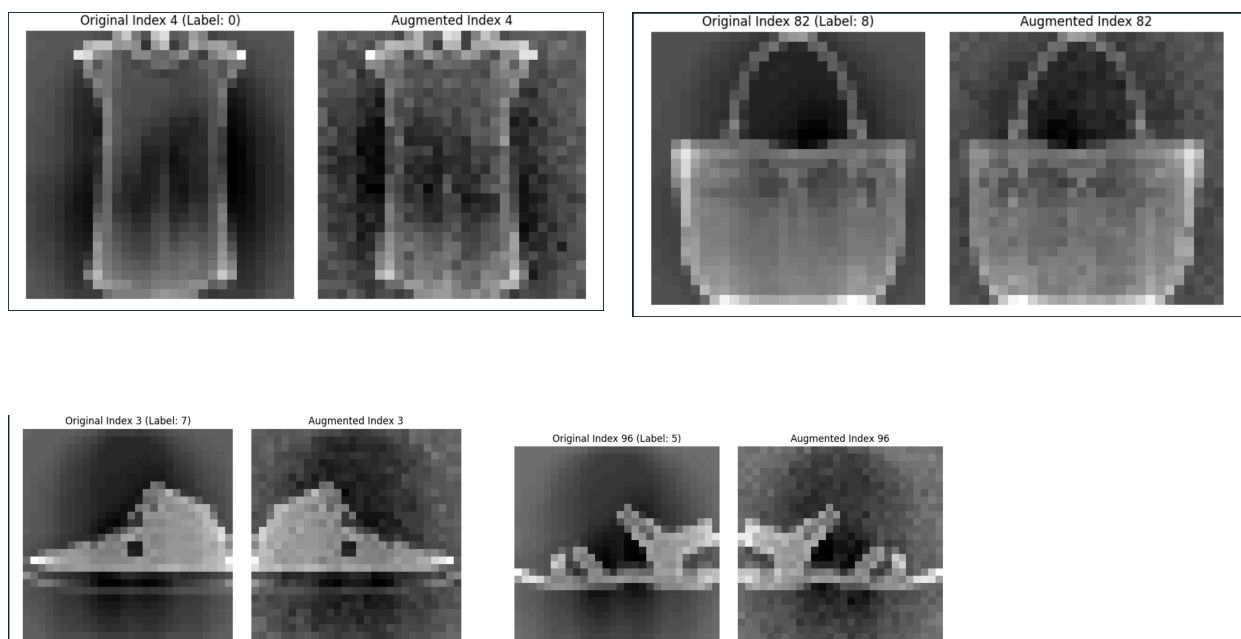
Using a random seed, we create indices for randomly selecting samples and assigning them into one of three new matrices. 10% of the samples are cut from the entire dataset and placed in the testing matrix. Another 10% are used for validation, and the rest is used for training. This split makes sense because training needs a sufficient amount of data to ensure the model does not overfit and start memorizing.

From there, the training data is split again between labeled and unlabeled data. While all the data is originally labeled, we will simply "forget" that the unlabeled data has the y label. The

split between the labeled and the unlabeled data is 20% and 80%, respectively. This choice makes sense for learning purposes as a way to simulate a scenario where you have an insufficient amount of labeled data. Instead of going to a specialist to create more labels, we use unlabeled data to provide more points of data, allowing the model to achieve better and more precise class separability.

Lastly, before the data is ready for the neural network, it must be augmented. Instead of taking a random percentage of the labeled samples, the augmentation was applied to a copy of all labeled training samples. These were then concatenated together, doubling the number of labeled training samples so that the final set is 50% original and 50% augmented.

The two augmentation techniques used are a horizontal flip and the addition of Gaussian noise. Using a Gaussian distribution allows the noise to blend in, which prevents the model from learning incorrect features while helping with generalization and preventing memorization. I chose a noise intensity of 0.15; this value provides a significant enough difference to challenge the model without breaking the structural relationships between the pixels too much. After the augmented samples are concatenated with the originals, they are shuffled to ensure that training batches are not composed solely of original or augmented samples.



Stage 2: Supervised Neural Network Baseline

The network is designed to have 2 hidden layers with an architecture described by the following: It has the input layer (the input vector of 784 normalized features of the image) connected to the first hidden layer. The first hidden layer is made up of 512 neurons, which are initialized using either a Random Normal distribution or a Random Uniform distribution. Each neuron multiplies the 784 features by its weight and adds the bias, after which it uses the Sigmoid as the activation function to produce the value that gets passed to the second hidden layer.

The second hidden layer is made up of 256 neurons that connect to the output layer, which consists of 10 neurons representing the classes: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle boot. The final output is determined by the Softmax function, which forces the probabilities to add up to 1. This allows the highest value to be our prediction while facilitating better backpropagation, as the cross-entropy loss can be calculated based on the probability distribution across all 10 classes.

The comparison between using a set small variance versus using Random Uniform shows that using the number of neurons in the layer (fanin) to scale the weights takes care of the problem where layers with more neurons become dominant. By adjusting the weights based on the layer size, we stop the signal from getting too weak or too strong as it passes through the network.

When looking at the results, the Random Uniform method started (at epoch 5) with a lower loss and higher accuracy. It ended with a better overall performance and, on top of that, finished much faster—completing in only 40 epochs compared to the 67 epochs it took when using Random Normal with small variance (seed #42).

Random normal with small variance
Epoch 5: Loss 1.0912, Acc 0.5570
Epoch 10: Loss 0.8036, Acc 0.6958
Epoch 15: Loss 0.6514, Acc 0.7612
Epoch 20: Loss 0.5847, Acc 0.7922
Epoch 25: Loss 0.5549, Acc 0.7992
Epoch 30: Loss 0.5231, Acc 0.8145
Epoch 35: Loss 0.5029, Acc 0.8223
Epoch 40: Loss 0.4835, Acc 0.8265
Epoch 45: Loss 0.4812, Acc 0.8308
Epoch 50: Loss 0.4780, Acc 0.8298
Epoch 55: Loss 0.4656, Acc 0.8370
Epoch 60: Loss 0.4582, Acc 0.8398
Epoch 65: Loss 0.4612, Acc 0.8418

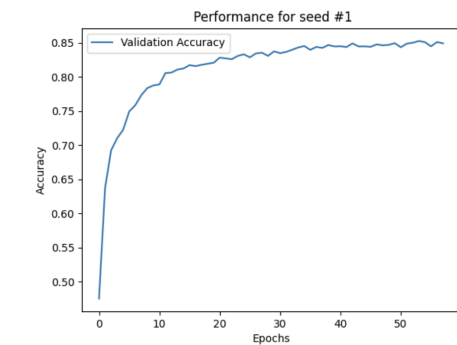
Random uniform
Epoch 5: Loss 0.7220, Acc 0.7403
Epoch 10: Loss 0.6117, Acc 0.7848
Epoch 15: Loss 0.5431, Acc 0.8078
Epoch 20: Loss 0.5144, Acc 0.8163
Epoch 25: Loss 0.5026, Acc 0.8158
Epoch 30: Loss 0.4806, Acc 0.8288
Epoch 35: Loss 0.4718, Acc 0.8368
Epoch 40: Loss 0.4579, Acc 0.8377

Results from running the model with 4 different seeds:

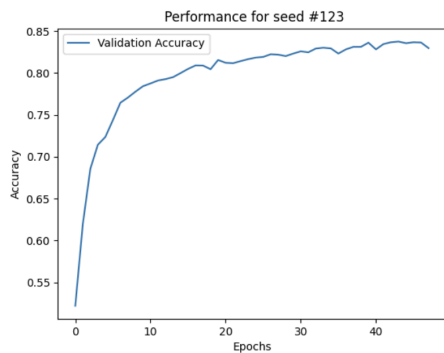
The results show a level of consistency that can be expected regardless of the seed, demonstrating that no matter where the model starts on the loss plane, it almost uniformly makes its way down the loss cavity and stops at around 84% accuracy and a loss of 0.45. An accuracy of 84% is much higher than I expected from a fully connected feed-forward network. Even without having information about the 2D relationships that pixels hold between themselves, the model—with enough samples and epochs—seems to have no problem separating the fashion dataset items into 10 classes. While this accuracy might fall short of a CNN, it resembles my own attempts at classifying these grayscale pieces of clothing and getting one or two out of ten wrong here and there. This is not too surprising, as sandals and shoes are hard to tell apart, and the same goes for dress and shirt items; the line is blurred without RGB colors, and we are mostly working with shapes and the proportions of those shapes.

Still, an 85% validation accuracy is a fantastic result and hints at the quality and size of the dataset used for training. That, along with the use of random uniform initialization for a good starting point and the use of softmax on the output to treat it as a multi-class problem, allows for

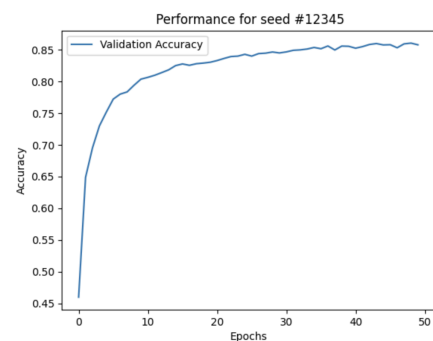
better entropy assignment and the backpropagation of better weight adjustments, leading to smooth convergence and a solid result for an FFN.2



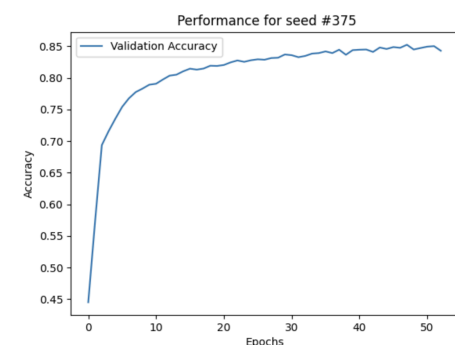
```
Epoch 5: Loss 0.7434, Acc 0.7262
Epoch 10: Loss 0.5989, Acc 0.7893
Epoch 15: Loss 0.5411, Acc 0.8130
Epoch 20: Loss 0.5151, Acc 0.8217
Epoch 25: Loss 0.4966, Acc 0.8333
Epoch 30: Loss 0.4814, Acc 0.8388
Epoch 35: Loss 0.4732, Acc 0.8463
Epoch 40: Loss 0.4673, Acc 0.8448
Epoch 45: Loss 0.4624, Acc 0.8448
Epoch 50: Loss 0.4600, Acc 0.8470
Epoch 55: Loss 0.4633, Acc 0.8502
Early stop at epoch 58
```



```
Loading dataset...
Epoch 5: Loss 0.7303, Acc 0.7237
Epoch 10: Loss 0.6096, Acc 0.7842
Epoch 15: Loss 0.5606, Acc 0.7998
Epoch 20: Loss 0.5238, Acc 0.8155
Epoch 25: Loss 0.5057, Acc 0.8183
Epoch 30: Loss 0.4937, Acc 0.8232
Epoch 35: Loss 0.4780, Acc 0.8293
Epoch 40: Loss 0.4694, Acc 0.8362
Epoch 45: Loss 0.4637, Acc 0.8355
Early stop at epoch 48
```



```
Epoch 5: Loss 0.6846, Acc 0.7518
Epoch 10: Loss 0.5653, Acc 0.8038
Epoch 15: Loss 0.5154, Acc 0.8252
Epoch 20: Loss 0.4881, Acc 0.8305
Epoch 25: Loss 0.4631, Acc 0.8430
Epoch 30: Loss 0.4545, Acc 0.8452
Epoch 35: Loss 0.4387, Acc 0.8538
Epoch 40: Loss 0.4259, Acc 0.8557
Epoch 45: Loss 0.4225, Acc 0.8578
Epoch 50: Loss 0.4298, Acc 0.8580
Early stop at epoch 50
```



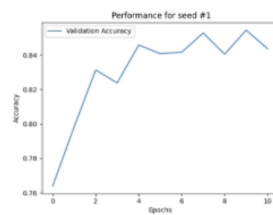
```
Loading dataset...
Epoch 5: Loss 0.7254, Acc 0.7355
Epoch 10: Loss 0.5911, Acc 0.7892
Epoch 15: Loss 0.5389, Acc 0.8103
Epoch 20: Loss 0.5110, Acc 0.8188
Epoch 25: Loss 0.4871, Acc 0.8278
Epoch 30: Loss 0.4725, Acc 0.8370
Epoch 35: Loss 0.4631, Acc 0.8392
Epoch 40: Loss 0.4585, Acc 0.8438
Epoch 45: Loss 0.4489, Acc 0.8457
Epoch 50: Loss 0.4437, Acc 0.8472
Early stop at epoch 53
```

Stage 3: Architecture Selection and Regularization

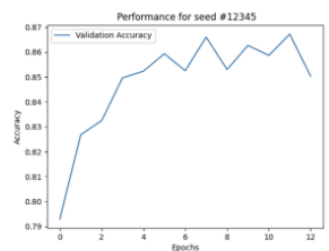
The decision to increase the learning rate from 0.01 to 0.2 (a 20x increase) was initially intended to provide enough "headroom" to implement Step Decay. The logic was to let the network cover a much larger search space by taking aggressive steps in the beginning and then refining those steps as accuracy improved.

To my surprise, simply increasing the learning rate allowed the network to reach the same accuracy baseline if not outperforming the model's previous results by 0.5 to 1% in a fraction of the time. The model went from an average of 40 epochs to convergence down to approximately 13 epochs

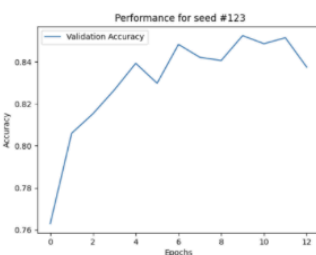
```
Loading dataset...
Epoch 2: Loss 0.5524, Acc 0.7985
Epoch 4: Loss 0.4774, Acc 0.8238
Epoch 6: Loss 0.4523, Acc 0.8408
Epoch 8: Loss 0.4497, Acc 0.8528
Epoch 10: Loss 0.4596, Acc 0.8545
Early stop at epoch 11
seed #1
```



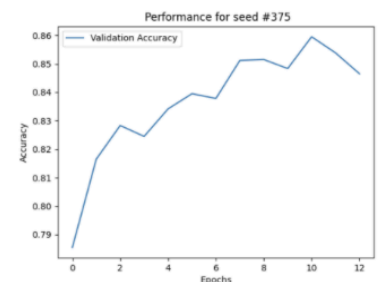
```
Loading dataset...
Epoch 2: Loss 0.4865, Acc 0.8268
Epoch 4: Loss 0.4229, Acc 0.8497
Epoch 6: Loss 0.4032, Acc 0.8593
Epoch 8: Loss 0.3991, Acc 0.8660
Epoch 10: Loss 0.4366, Acc 0.8627
Epoch 12: Loss 0.4462, Acc 0.8672
Early stop at epoch 13
seed #12345
```



```
Loading dataset...
Epoch 2: Loss 0.5344, Acc 0.8015
Epoch 4: Loss 0.4715, Acc 0.8245
Epoch 6: Loss 0.4896, Acc 0.8395
Epoch 8: Loss 0.4585, Acc 0.8512
Epoch 10: Loss 0.4597, Acc 0.8483
Epoch 12: Loss 0.4841, Acc 0.8538
Early stop at epoch 13
seed #123
```



```
Loading dataset...
Epoch 2: Loss 0.5061, Acc 0.8165
Epoch 4: Loss 0.4975, Acc 0.8245
Epoch 6: Loss 0.4566, Acc 0.8395
Epoch 8: Loss 0.4412, Acc 0.8512
Epoch 10: Loss 0.4350, Acc 0.8483
Epoch 12: Loss 0.4752, Acc 0.8538
Early stop at epoch 13
seed #375
```



(LEARNING RATE INCREASED 20X RESULTS)

The addition of a step decay provided notable differences in the model's training dynamics, particularly when reducing the step size by multiplying it by 0.9 or 0.7 every two epochs. If the frequency of the step decay is decreased too much—to once every five epochs or more—the step reduction isn't fast enough, and the network converges prematurely because the necessary

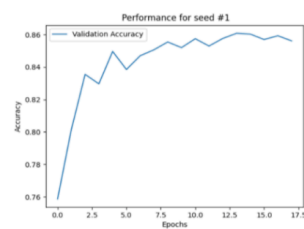
smaller adjustments arrive too late to make a functional difference. In these cases, where the curve remains smooth, the model effectively isolates more than is optimal. Conversely, if the size of the decay is too aggressive, such as a 0.5 multiplier, the system may fail to converge effectively before reaching the maximum number of epochs in the loop, resulting in no measurable improvement in accuracy.

Ultimately, getting the right frequency of the decay as well as the correct size of the step determines if the adaptive learning rate provides any benefit at all. When the right combination of initial learning rate, decay frequency, and step size changes are found, there are visible, albeit small, results; the primary benefit is a smoother increase in accuracy and a gain of around +1% in the model's final accuracy. This improvement is technically significant as it indicates the model is fine-tuning its weights within the "loss cavity" rather than oscillating around it. However, if these variables are not tuned properly, there is either no measurable gain or even a loss to the final accuracy, depending on how mismatched the variables are to the specific optimization landscape of the dataset.

```

Loading dataset...
Epoch 2: Loss 0.5488, Acc 0.8015
--- Learning rate dropped to 0.13999999999999999 ---
Epoch 4: Loss 0.4726, Acc 0.8297
--- Learning rate dropped to 0.09799999999999999 ---
Epoch 6: Loss 0.4082, Acc 0.8385
--- Learning rate dropped to 0.0686 ---
Epoch 8: Loss 0.4322, Acc 0.8507
--- Learning rate dropped to 0.04801999999999999 ---
Epoch 10: Loss 0.4352, Acc 0.8520
--- Learning rate dropped to 0.03361399999999999 ---
Epoch 12: Loss 0.4503, Acc 0.8530
--- Learning rate dropped to 0.023529799999999993 ---
Epoch 14: Loss 0.4301, Acc 0.8608
--- Learning rate dropped to 0.016470859999999993 ---
Epoch 16: Loss 0.4388, Acc 0.8570
--- Learning rate dropped to 0.011529601999999995 ---
Epoch 18: Loss 0.4433, Acc 0.8562
Early stop at epoch 18
seed #1

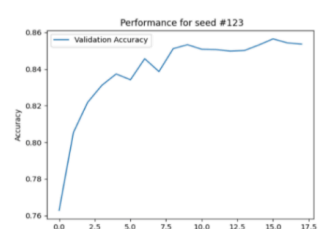
```



```

Epoch 2: Loss 0.5371, Acc 0.8053
--- Learning rate dropped to 0.13999999999999999 ---
Epoch 4: Loss 0.4882, Acc 0.8312
--- Learning rate dropped to 0.09799999999999999 ---
Epoch 6: Loss 0.4701, Acc 0.8342
--- Learning rate dropped to 0.0686 ---
Epoch 8: Loss 0.4503, Acc 0.8387
--- Learning rate dropped to 0.04801999999999999 ---
Epoch 10: Loss 0.4332, Acc 0.8533
--- Learning rate dropped to 0.03361399999999999 ---
Epoch 12: Loss 0.4381, Acc 0.8507
--- Learning rate dropped to 0.023529799999999993 ---
Epoch 14: Loss 0.4361, Acc 0.8502
--- Learning rate dropped to 0.016470859999999993 ---
Epoch 16: Loss 0.4369, Acc 0.8568
--- Learning rate dropped to 0.011529601999999995 ---
Epoch 18: Loss 0.4436, Acc 0.8537
Early stop at epoch 18
seed #123

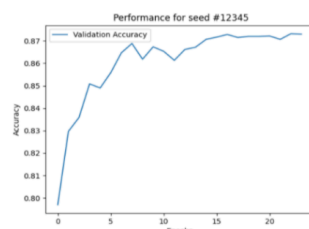
```



```

Loading dataset...
Epoch 2: Loss 0.4820, Acc 0.8297
--- Learning rate dropped to 0.13999999999999999 ---
Epoch 4: Loss 0.4215, Acc 0.8588
--- Learning rate dropped to 0.09799999999999999 ---
Epoch 6: Loss 0.4075, Acc 0.8558
--- Learning rate dropped to 0.0686 ---
Epoch 8: Loss 0.3925, Acc 0.8688
--- Learning rate dropped to 0.04801999999999999 ---
Epoch 10: Loss 0.3909, Acc 0.8673
--- Learning rate dropped to 0.03361399999999999 ---
Epoch 12: Loss 0.4200, Acc 0.8613
--- Learning rate dropped to 0.023529799999999993 ---
Epoch 14: Loss 0.3999, Acc 0.8672
--- Learning rate dropped to 0.016470859999999993 ---
Epoch 16: Loss 0.3979, Acc 0.8717
--- Learning rate dropped to 0.011529601999999995 ---
Epoch 18: Loss 0.3972, Acc 0.8715
--- Learning rate dropped to 0.008070721399999996 ---
Epoch 20: Loss 0.4003, Acc 0.8720
--- Learning rate dropped to 0.005649504979999997 ---
Epoch 22: Loss 0.4060, Acc 0.8707
--- Learning rate dropped to 0.003954653485999997 ---
Epoch 24: Loss 0.4081, Acc 0.8730
Early stop at epoch 24
seed #12345

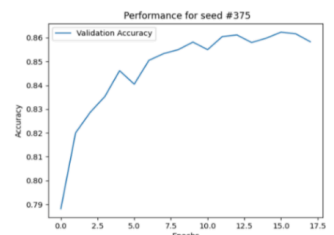
```



```

Loading dataset...
Epoch 2: Loss 0.5029, Acc 0.8200
--- Learning rate dropped to 0.13999999999999999 ---
Epoch 4: Loss 0.4711, Acc 0.8353
--- Learning rate dropped to 0.09799999999999999 ---
Epoch 6: Loss 0.4486, Acc 0.8495
--- Learning rate dropped to 0.0686 ---
Epoch 8: Loss 0.4199, Acc 0.8533
--- Learning rate dropped to 0.04801999999999999 ---
Epoch 10: Loss 0.4159, Acc 0.8582
--- Learning rate dropped to 0.03361399999999999 ---
Epoch 12: Loss 0.4161, Acc 0.8605
--- Learning rate dropped to 0.023529799999999993 ---
Epoch 14: Loss 0.4254, Acc 0.8580
--- Learning rate dropped to 0.016470859999999993 ---
Epoch 16: Loss 0.4246, Acc 0.8623
--- Learning rate dropped to 0.011529601999999995 ---
Epoch 18: Loss 0.4334, Acc 0.8583
Early stop at epoch 18
seed #375

```



(STATE DECAY BY A FACTOR RESULTS)

The addition of step decay introduced a significant trade-off in training efficiency, as the model required approximately 30% more time—nearly doubling the number of epochs—to reach convergence compared to the fixed high-learning rate baseline. Despite this increase in training time, the resulting accuracy curve was noticeably smoother. Without the adjustment provided by step decay, the model typically began to oscillate aggressively after the first five epochs, as the large steps caused the weights to bounce back and forth across the narrowest parts of the loss cavity. By gradually reducing the learning rate, the step decay dampened these volatile "up and down" fluctuations, allowing the network to settle into a more stable and precise local minimum.

Dropout Regularization

This technique, like most, provides significant benefits only when it is tuned correctly. For the base network (using seed #1), the performance sits at roughly 85% accuracy. When the model gets to numbers above 80%, every 1% improvement is critical. This makes the 88% accuracy achieved with a dropout rate of 0.2 that much more impressive, as it outperforms the baseline network by a full 3%. This shows that if we don't overdo it by zeroing out too many connections, the randomness introduced by dropout makes serious improvements to both accuracy and loss by preventing the model from overfitting.

However, testing the extremes—rates of 0.1 and 0.5—reveals exactly how this tuning impacts the network's tendency to memorize versus generalize.

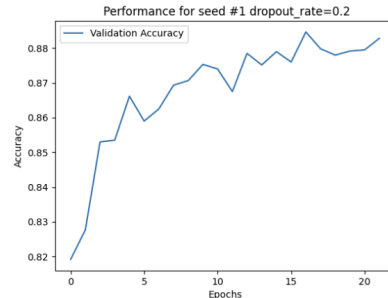
When the dropout rate was set to a low 0.1, the model also reached 88% accuracy. But looking closely at the metrics, the validation loss eventually rebounded, creeping up almost 10% from its lowest point. This upward trend indicates that a 0.1 rate is actually not doing enough to fight

memorization; the network eventually starts overfitting to the training data, even if the accuracy number looks successful.

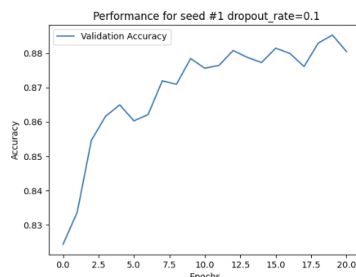
On the other hand, an aggressive dropout rate of 0.5 tells a different story. Initially, it might seem like zeroing out half the connections causes the network to forget too much to learn properly. However, it actually reached a highly respectable 87% accuracy. More importantly, its validation loss was monotonically decreasing the entire time. While dropping half the connections causes noticeable oscillations and swings in the accuracy curve, the smooth, consistently decreasing loss proves that 0.5 is highly effective at fighting memorization.

Ultimately, the 0.1 rate fails to regularize enough, and the 0.5 rate forces the network to work a bit too hard through randomness. Finding that middle ground—like the 0.2 rate—provides the perfect balance, yielding the highest accuracy without sacrificing the health of the loss curve.

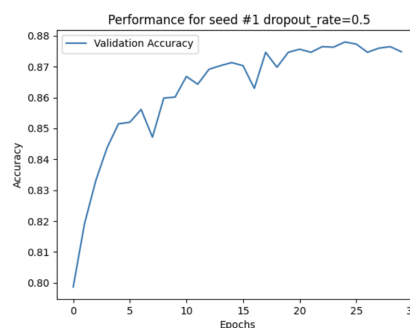
```
Epoch 2: Loss 0.4666, Acc 0.8277
Epoch 4: Loss 0.3943, Acc 0.8535
Epoch 6: Loss 0.3921, Acc 0.8590
Epoch 8: Loss 0.3554, Acc 0.8693
Epoch 10: Loss 0.3560, Acc 0.8753
Epoch 12: Loss 0.3628, Acc 0.8675
Epoch 14: Loss 0.3535, Acc 0.8752
Epoch 16: Loss 0.3647, Acc 0.8760
Epoch 18: Loss 0.3472, Acc 0.8798
Epoch 20: Loss 0.3537, Acc 0.8792
Epoch 22: Loss 0.3789, Acc 0.8828
Early stop at epoch 22
seed #1
```



```
Epoch 2: Loss 0.4506, Acc 0.8337
Epoch 4: Loss 0.3791, Acc 0.8617
Epoch 6: Loss 0.3926, Acc 0.8603
Epoch 8: Loss 0.3421, Acc 0.8720
Epoch 10: Loss 0.3519, Acc 0.8785
Epoch 12: Loss 0.3495, Acc 0.8765
Epoch 14: Loss 0.3718, Acc 0.8788
Epoch 16: Loss 0.3560, Acc 0.8815
Epoch 18: Loss 0.3798, Acc 0.8762
Epoch 20: Loss 0.4009, Acc 0.8853
Early stop at epoch 21
seed #1
```



```
Loading dataset...
Epoch 2: Loss 0.4824, Acc 0.8192
Epoch 4: Loss 0.4205, Acc 0.8438
Epoch 6: Loss 0.4044, Acc 0.8520
Epoch 8: Loss 0.4021, Acc 0.8472
Epoch 10: Loss 0.3797, Acc 0.8602
Epoch 12: Loss 0.3567, Acc 0.8643
Epoch 14: Loss 0.3563, Acc 0.8703
Epoch 16: Loss 0.3471, Acc 0.8703
Epoch 18: Loss 0.3384, Acc 0.8747
Epoch 20: Loss 0.3415, Acc 0.8747
Epoch 22: Loss 0.3426, Acc 0.8747
Epoch 24: Loss 0.3405, Acc 0.8763
Epoch 26: Loss 0.3324, Acc 0.8773
Epoch 28: Loss 0.3395, Acc 0.8760
Epoch 30: Loss 0.3460, Acc 0.8748
Early stop at epoch 30
seed #1
```



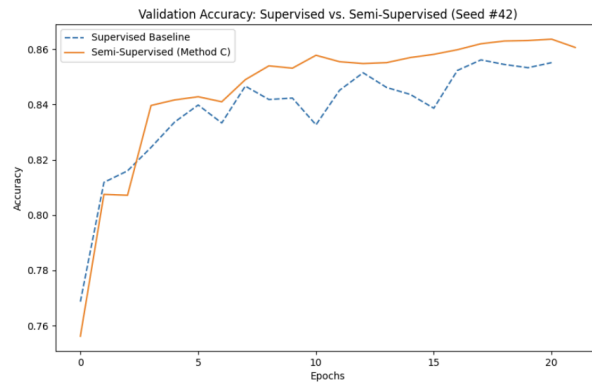
Stage 4: Method C: Consistency regularization with Data Augmentation

--- Training Supervised Model ---

Epoch 2: Loss 0.5287, Acc 0.8118
Epoch 4: Loss 0.4765, Acc 0.8245
Epoch 6: Loss 0.4465, Acc 0.8398
Epoch 8: Loss 0.4352, Acc 0.8467
Epoch 10: Loss 0.4418, Acc 0.8423
Epoch 12: Loss 0.4304, Acc 0.8452
Epoch 14: Loss 0.4306, Acc 0.8462
Epoch 16: Loss 0.4345, Acc 0.8387
Epoch 18: Loss 0.4191, Acc 0.8562
Epoch 20: Loss 0.4176, Acc 0.8533
Early stop at epoch 21

--- Training Semi-Supervised Model ---

Epoch 2: Val Loss 0.5337, Val Acc 0.8075 | Lambda: 0.01
Epoch 4: Val Loss 0.4536, Val Acc 0.8397 | Lambda: 0.03
Epoch 6: Val Loss 0.4371, Val Acc 0.8428 | Lambda: 0.05
Epoch 8: Val Loss 0.4303, Val Acc 0.8490 | Lambda: 0.07
Epoch 10: Val Loss 0.4224, Val Acc 0.8532 | Lambda: 0.09
Epoch 12: Val Loss 0.4292, Val Acc 0.8555 | Lambda: 0.11
Epoch 14: Val Loss 0.4242, Val Acc 0.8552 | Lambda: 0.13
Epoch 16: Val Loss 0.4208, Val Acc 0.8582 | Lambda: 0.15
Epoch 18: Val Loss 0.4394, Val Acc 0.8620 | Lambda: 0.17
Epoch 20: Val Loss 0.4345, Val Acc 0.8632 | Lambda: 0.19
Epoch 22: Val Loss 0.4499, Val Acc 0.8607 | Lambda: 0.21
Early stop at epoch 22

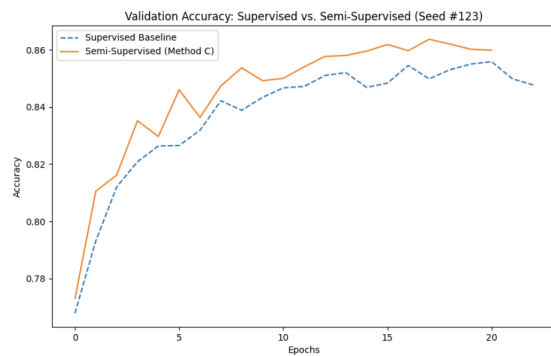


--- Training Supervised Model ---

Epoch 2: Loss 0.5752, Acc 0.7932
Epoch 4: Loss 0.5100, Acc 0.8208
Epoch 6: Loss 0.4662, Acc 0.8265
Epoch 8: Loss 0.4388, Acc 0.8422
Epoch 10: Loss 0.4370, Acc 0.8433
Epoch 12: Loss 0.4269, Acc 0.8472
Epoch 14: Loss 0.4466, Acc 0.8520
Epoch 16: Loss 0.4330, Acc 0.8483
Epoch 18: Loss 0.4434, Acc 0.8498
Epoch 20: Loss 0.4154, Acc 0.8550
Epoch 22: Loss 0.4356, Acc 0.8498
Early stop at epoch 23

--- Training Semi-Supervised Model ---

Epoch 2: Val Loss 0.5147, Val Acc 0.8105 | Lambda: 0.01
Epoch 4: Val Loss 0.4715, Val Acc 0.8352 | Lambda: 0.03
Epoch 6: Val Loss 0.4384, Val Acc 0.8460 | Lambda: 0.05
Epoch 8: Val Loss 0.4291, Val Acc 0.8473 | Lambda: 0.07
Epoch 10: Val Loss 0.4252, Val Acc 0.8492 | Lambda: 0.09
Epoch 12: Val Loss 0.4235, Val Acc 0.8540 | Lambda: 0.11
Epoch 14: Val Loss 0.4203, Val Acc 0.8580 | Lambda: 0.13
Epoch 16: Val Loss 0.4394, Val Acc 0.8618 | Lambda: 0.15
Epoch 18: Val Loss 0.4270, Val Acc 0.8637 | Lambda: 0.17
Epoch 20: Val Loss 0.4434, Val Acc 0.8602 | Lambda: 0.19
Early stop at epoch 21

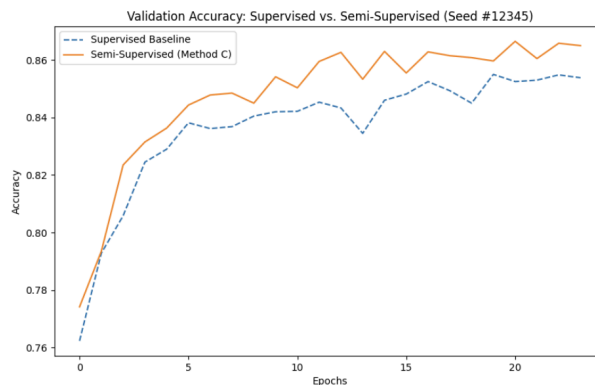


--- Training Supervised Model ---

Epoch 2: Loss 0.5530, Acc 0.7928
Epoch 4: Loss 0.4845, Acc 0.8245
Epoch 6: Loss 0.4553, Acc 0.8382
Epoch 8: Loss 0.4578, Acc 0.8368
Epoch 10: Loss 0.4438, Acc 0.8420
Epoch 12: Loss 0.4308, Acc 0.8453
Epoch 14: Loss 0.4618, Acc 0.8345
Epoch 16: Loss 0.4349, Acc 0.8482
Epoch 18: Loss 0.4342, Acc 0.8493
Epoch 20: Loss 0.4231, Acc 0.8550
Epoch 22: Loss 0.4374, Acc 0.8530
Epoch 24: Loss 0.4438, Acc 0.8538
Early stop at epoch 24

--- Training Semi-Supervised Model ---

Epoch 2: Val Loss 0.5538, Val Acc 0.7935 | Lambda: 0.01
Epoch 4: Val Loss 0.4739, Val Acc 0.8315 | Lambda: 0.03
Epoch 6: Val Loss 0.4455, Val Acc 0.8443 | Lambda: 0.05
Epoch 8: Val Loss 0.4283, Val Acc 0.8485 | Lambda: 0.07
Epoch 10: Val Loss 0.4277, Val Acc 0.8542 | Lambda: 0.09
Epoch 12: Val Loss 0.4236, Val Acc 0.8595 | Lambda: 0.11
Epoch 14: Val Loss 0.4279, Val Acc 0.8533 | Lambda: 0.13
Epoch 16: Val Loss 0.4483, Val Acc 0.8555 | Lambda: 0.15
Epoch 18: Val Loss 0.4267, Val Acc 0.8615 | Lambda: 0.17
Epoch 20: Val Loss 0.4400, Val Acc 0.8597 | Lambda: 0.19
Epoch 22: Val Loss 0.4359, Val Acc 0.8605 | Lambda: 0.21
Epoch 24: Val Loss 0.4461, Val Acc 0.8650 | Lambda: 0.23
Early stop at epoch 24



To effectively utilize the 80% of the training data that lacks labels, the training objective was modified to include a consistency term alongside the supervised loss. The supervised loss is calculated strictly on the 20% labeled data, teaching the network the fundamental mappings between pixel features and the clothing classes. The consistency loss acts as a penalty applied to the unlabeled data, forcing the model to produce stable, identical predictions for an image even when it is randomly altered. In the training loop, the model first processes a batch of labeled data to calculate the supervised loss. Simultaneously, a batch of unlabeled data is sampled and a physical copy is created using the flipping and noise techniques established earlier, performing a forward pass on both the clean images and their noisy twins.

In order to calculate this consistency penalty, the outputs from both unlabeled passes are converted into probabilities. The prediction on the clean image is frozen as a fixed target, ensuring the network is only penalized if the noisy prediction drifts away from this clean baseline. The difference between these two distributions becomes the consistency loss. To prevent the network from being overwhelmed by this penalty before it learns basic features from the labeled data, a linear warm-up schedule gradually scales the consistency loss from 0 to 1 over the training process before adding it to the total loss. Finally, to ensure a fair comparison between the baseline and this semi-supervised pipeline, a reset function was used. This function re-initializes the network's weights to a fresh uniform distribution, clears the momentum trackers, and locks the random seeds before every run, guaranteeing both models start with the exact same initial conditions and process data in the exact same order.

The initial results across three different seeds demonstrated that the supervised baseline achieved an accuracy of approximately 85%, while the semi-supervised model improved upon this to reach 86%. This outcome aligns with expectations; because only 20% of the training data was labeled, the consistency regularization was able to effectively leverage the remaining 80% of the unlabeled pool to smooth out the model's decision boundaries. To explore this dynamic

Looking at the performance graphs, both the supervised and semi-supervised models start off with nearly identical validation accuracy. However, their paths diverge pretty quickly. After just one or two

epochs, the semi-supervised model begins to show clear improvements over the baseline. By epoch 5, this gap widens to about a 1% advantage. What stands out is that this improvement isn't just a temporary jump; the semi-supervised model maintains that consistent 1% lead throughout the rest of the training process until both models finally reach convergence.

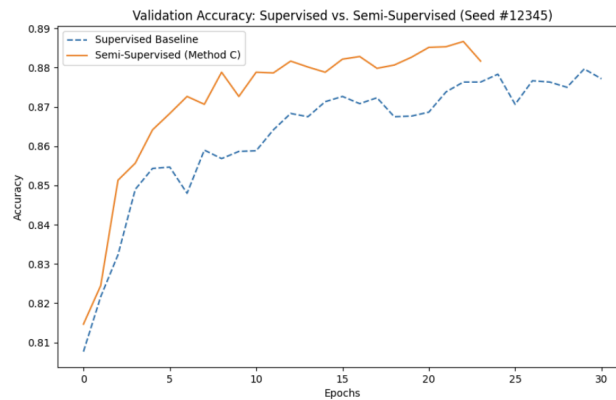
```

--- Training Supervised Model ---
Epoch 2: Loss 0.4792, Acc 0.8217
Epoch 4: Loss 0.4234, Acc 0.8490
Epoch 6: Loss 0.3923, Acc 0.8547
Epoch 8: Loss 0.3758, Acc 0.8590
Epoch 10: Loss 0.3629, Acc 0.8587
Epoch 12: Loss 0.3592, Acc 0.8642
Epoch 14: Loss 0.3625, Acc 0.8675
Epoch 16: Loss 0.3426, Acc 0.8727
Epoch 18: Loss 0.3480, Acc 0.8723
Epoch 20: Loss 0.3344, Acc 0.8677
Epoch 22: Loss 0.3356, Acc 0.8738
Epoch 24: Loss 0.3347, Acc 0.8763
Epoch 26: Loss 0.3426, Acc 0.8707
Epoch 28: Loss 0.3286, Acc 0.8763
Epoch 30: Loss 0.3392, Acc 0.8797
Early stop at epoch 31

--- Training Semi-Supervised Model ---
Epoch 2: Val Loss 0.4748, Val Acc 0.8245 | Lambda: 0.01
Epoch 4: Val Loss 0.3928, Val Acc 0.8557 | Lambda: 0.03
Epoch 6: Val Loss 0.3587, Val Acc 0.8683 | Lambda: 0.05
Epoch 8: Val Loss 0.3534, Val Acc 0.8707 | Lambda: 0.07
Epoch 10: Val Loss 0.3422, Val Acc 0.8727 | Lambda: 0.09
Epoch 12: Val Loss 0.3402, Val Acc 0.8787 | Lambda: 0.11
Epoch 14: Val Loss 0.3300, Val Acc 0.8802 | Lambda: 0.13
Epoch 16: Val Loss 0.3355, Val Acc 0.8822 | Lambda: 0.15
Epoch 18: Val Loss 0.3394, Val Acc 0.8798 | Lambda: 0.17
Epoch 20: Val Loss 0.3428, Val Acc 0.8827 | Lambda: 0.19
Epoch 22: Val Loss 0.3411, Val Acc 0.8853 | Lambda: 0.21
Epoch 24: Val Loss 0.3498, Val Acc 0.8817 | Lambda: 0.23
Early stop at epoch 24

>>> FINAL TEST ACCURACY: 0.8855 <<<

```

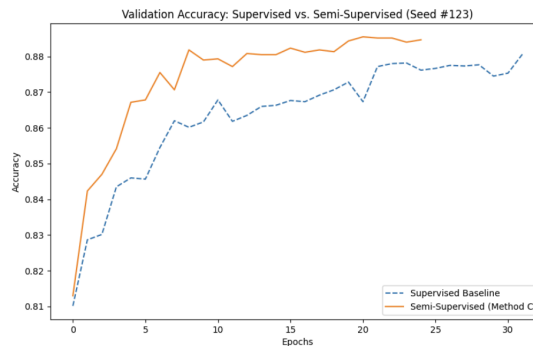


```

--- Training Supervised Model ---
Epoch 2: Loss 0.4604, Acc 0.8287
Epoch 4: Loss 0.4195, Acc 0.8435
Epoch 6: Loss 0.4056, Acc 0.8457
Epoch 8: Loss 0.3779, Acc 0.8620
Epoch 10: Loss 0.3685, Acc 0.8617
Epoch 12: Loss 0.3730, Acc 0.8618
Epoch 14: Loss 0.3586, Acc 0.8660
Epoch 16: Loss 0.3582, Acc 0.8677
Epoch 18: Loss 0.3475, Acc 0.8692
Epoch 20: Loss 0.3451, Acc 0.8728
Epoch 22: Loss 0.3479, Acc 0.8772
Epoch 24: Loss 0.3383, Acc 0.8782
Epoch 26: Loss 0.3402, Acc 0.8767
Epoch 28: Loss 0.3404, Acc 0.8773
Epoch 30: Loss 0.3406, Acc 0.8745
Epoch 32: Loss 0.3439, Acc 0.8807
Early stop at epoch 32

--- Training Semi-Supervised Model ---
Epoch 2: Val Loss 0.4279, Val Acc 0.8423 | Lambda: 0.01
Epoch 4: Val Loss 0.3967, Val Acc 0.8542 | Lambda: 0.03
Epoch 6: Val Loss 0.3594, Val Acc 0.8678 | Lambda: 0.05
Epoch 8: Val Loss 0.3446, Val Acc 0.8707 | Lambda: 0.07
Epoch 10: Val Loss 0.3351, Val Acc 0.8790 | Lambda: 0.09
Epoch 12: Val Loss 0.3404, Val Acc 0.8772 | Lambda: 0.11
Epoch 14: Val Loss 0.3319, Val Acc 0.8805 | Lambda: 0.13
Epoch 16: Val Loss 0.3438, Val Acc 0.8823 | Lambda: 0.15
Epoch 18: Val Loss 0.3347, Val Acc 0.8818 | Lambda: 0.17
Epoch 20: Val Loss 0.3273, Val Acc 0.8843 | Lambda: 0.19
Epoch 22: Val Loss 0.3379, Val Acc 0.8852 | Lambda: 0.21
Epoch 24: Val Loss 0.3393, Val Acc 0.8840 | Lambda: 0.23
Early stop at epoch 25

```

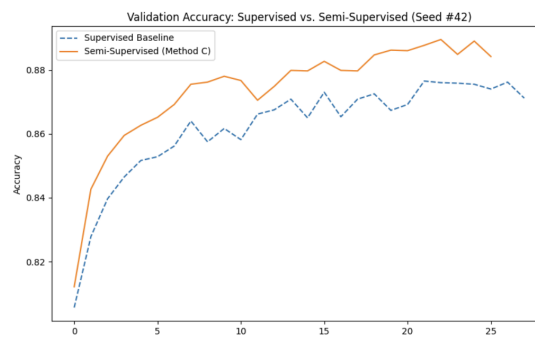


```

--- Training Supervised Model ---
Epoch 2: Loss 0.4616, Acc 0.8278
Epoch 4: Loss 0.4147, Acc 0.8465
Epoch 6: Loss 0.3927, Acc 0.8528
Epoch 8: Loss 0.3852, Acc 0.8648
Epoch 10: Loss 0.3633, Acc 0.8617
Epoch 12: Loss 0.3623, Acc 0.8652
Epoch 14: Loss 0.3531, Acc 0.8788
Epoch 16: Loss 0.3485, Acc 0.8738
Epoch 18: Loss 0.3512, Acc 0.8780
Epoch 20: Loss 0.3616, Acc 0.8673
Epoch 22: Loss 0.3426, Acc 0.8765
Epoch 24: Loss 0.3472, Acc 0.8758
Epoch 26: Loss 0.3399, Acc 0.8740
Epoch 28: Loss 0.3582, Acc 0.8712
Early stop at epoch 28

--- Training Semi-Supervised Model ---
Epoch 2: Val Loss 0.4403, Val Acc 0.8427 | Lambda: 0.01
Epoch 4: Val Loss 0.3811, Val Acc 0.8595 | Lambda: 0.03
Epoch 6: Val Loss 0.3638, Val Acc 0.8652 | Lambda: 0.05
Epoch 8: Val Loss 0.3482, Val Acc 0.8755 | Lambda: 0.07
Epoch 10: Val Loss 0.3375, Val Acc 0.8780 | Lambda: 0.09
Epoch 12: Val Loss 0.3358, Val Acc 0.8785 | Lambda: 0.11
Epoch 14: Val Loss 0.3343, Val Acc 0.8790 | Lambda: 0.13
Epoch 16: Val Loss 0.3290, Val Acc 0.8827 | Lambda: 0.15
Epoch 18: Val Loss 0.3429, Val Acc 0.8797 | Lambda: 0.17
Epoch 20: Val Loss 0.3357, Val Acc 0.8862 | Lambda: 0.19
Epoch 22: Val Loss 0.3420, Val Acc 0.8877 | Lambda: 0.21
Epoch 24: Val Loss 0.3425, Val Acc 0.8848 | Lambda: 0.23
Epoch 26: Val Loss 0.3510, Val Acc 0.8842 | Lambda: 0.25
Early stop at epoch 26

```



In conclusion, building this feed-forward network from scratch for the Fashion-MNIST dataset highlighted exactly how far architectural tuning and clever regularization can take a model that inherently lacks spatial awareness. Implementing consistency regularization provided a clear improvement in test accuracy, successfully pushing the model from a baseline of 85% to 86% when operating under a strictly limited 20/80 labeled-to-unlabeled split. Observing the training dynamics, this improvement materialized surprisingly fast; rather than slowly compounding over time, the semi-supervised model established a distinct 1% accuracy advantage within the first five epochs and simply maintained that stable lead until convergence. However, altering the environment also revealed the distinct failure modes of this approach. When I increased the labeled data pool to a 50/50 split, both models jumped to around 88% accuracy, but the semi-supervised advantage essentially vanished—even performing identically to the baseline on seed 123. Which leads me to say that, while semi-supervised learning mostly proves to be a highly effective tool for data-scarce environments(1%is still a lot), its power drops off significantly the moment labeled data is no longer limited. Much like dropout and step decay, it only provides a real functional benefit when its application is explicitly matched to the actual limitations of the dataset.

	Configuration	Test Accur	Improvement	
	Baseline (10% labeled	0.8427		
	+ Preprocessing (Z-Sco	0.8542	1.15%	
	+ Better Initialization (R	0.8678	1.36%	
	Step Decay	0.8787	1.09%	
	Dropout (without stepDecay)	0.879	1.10%	(without counting stepDecay
	 mi-Supervised Lea	0.8855	0.65%	
		0.8855	+4.28% Total	